BÖLÜM 8: GÖZLEMLENEBILIRLIK VE OPERASYONEL MÜKEMMELLIK — KARANLIKTA GÖRMEK

💣 9 dk • DERS 29

Parça 29: Gözlemlenebilirlik Üçlüsü: Metrikler, Loglar ve Dağıtık İzleme (Tracing)

OpenTelemetry bağlam aktarımı ile 15 mikroservis arasındaki 200 milisaniyelik yavaşlığı 30 saniyede bulmak.

🔥 SAHADAN YAŞANMIŞ KRİZ HİKAYESİ

Bir online aracı kurumun 40 mikroservisi vardı. Borsa açılışında hisse senedi alım emri vermek 150 milisaniye yerine 8 saniye sürmeye başladı. Operasyon ekibi saniyede 50.000 satır yapılandırılmamış metin logu akan 40 ayrı terminal penceresine boş gözlerle bakıyordu. Her servis aynı şeyi yazıyordu: `Bağlantı zaman aşımı` veya `Alt servisten yanıt bekleniyor`. Ama suçlu kimdi? Ekipler 4 saat boyunca tartıştı: Emir ekibi hesap ekibini, hesap ekibi risk ekibini, risk ekibi veritabanı ekibini suçladı. Mimar nihayet sisteme tek bir `trace_id` ile OpenTelemetry dağıtık izleme kurduğunda gerçek 30 saniyede ortaya çıktı: Junior bir mühendisin yazdığı unutulmuş bir IP sorgu servisi, her hisse alımında aracı kurumun kendi IP adresini ters-DNS ile sorgulamaya çalışıyordu!

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

İzleme (Monitoring) sistem çöktüğünde size haber verir ('CPU %95'); Gözlemlenebilirlik (Observability) ise sistemin dış çıktılarına bakarak iç durumunu anlamanızı ve daha önce aklınıza gelmemiş soruları bile sorabilmenizi sağlar. Dağıtık bir mimaride `grep` ile hata arayamazsınız. İhtiyacınız olan **Gözlemlenebilirlik Üçlüsü**'dür: Metrikler yangın olduğunu haber verir, Loglar dumanın kokusunu tarif eder, Dağıtık İzleme (Tracing) ise yangını başlatan kibriti gösterir.

Temel Mimari İlkeler ve Zihinsel Modeller

Gözlemlenebilirlik Üçlüsünün Rolleri

- **Metrikler (Prometheus, Datadog):** Zaman içindeki sayısal veriler (sayaçlar, göstergeler). Depolaması çok ucuzdur, alarmlar için harikadır ama tekil istek bağlamı içermez.
- **Yapılandırılmış Loglar (JSON):** Detaylı olay kayıtları. Zaman damgası, seviye ve bağlam (`kullanici_id`, `islem_id`) içermelidir.
- **Dağıtık İzleme (OpenTelemetry, Jaeger):** Bir isteği ağ sınırları boyunca adım adım takip eder. İsteğe benzersiz bir `trace_id` atanır ve her sekmede bir `span` oluşturulur. Gecikmeler şelale grafiğinde net şekilde görünür.

Metriklerde Kardinalite Tuzağı

Metrik etiketlerine asla `kullanici_id` veya `siparis_id` gibi yüksek kardinaliteli (milyonlarca farklı değer alabilen) alanlar eklemeyin! Prometheus'un belleğini patlatır ve SaaS faturanıza binlerce dolar yansıtır. Yüksek kardinalite log ve trace içindir.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Düz Metin Loglama:** `console.log('Kullanıcı girdi: ' + id)` yazarak logları aranamaz hale getirmek (Bunun yerine JSON log kullanın).

- **Kopuk İzleme Zinciri:** Aşağı servise HTTP isteği atarken W3C `traceparent` başlığını iletmeyi unutup dağıtık izleme şelalesini parçalamak.
- **Alarm Yorgunluğu (Yalancı Çoban):** Önemsiz her uyarıya PagerDuty alarmı bağlayıp mühendisleri gece uyarıları sessize almaya alıştırmak.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Bütün servislerde üretici bağımsız standart olarak OpenTelemetry (OTel) SDK'sını kullan.
> - **2. Adım:** API Gateway'de her gelen isteğe bir `trace_id` üret ve yanıt başlığına (`X-Trace-Id`) ekle.
> - **3. Adım:** Bütün uygulama loglarını JSON formatına çevir ve o anki aktif `trace_id`yi her log satırına otomatik iliştir.

Mimari Çıkarımlar ve Hile Tablosu

- Dağıtık izleme olmadan mikroservis debug etmek, 40 ayrı samanlıkta iğne aramaya benzer.
- Metrikler sorunu haber verir; izler (traces) yerini gösterir; loglar sebebini açıklar.
- Metrik etiketlerinde düşük kardinalite; log ve izlerde yüksek kardinalite kullanın.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Canlıdaki tek bir kullanıcı isteğini alıp yaptığı tüm veritabanı sorgularını ve mikroservis seyahatlerini baştan sona tek bir ekranda görebiliyor musunuz? Eksik olan nedir?